

Day 11: Networking fundamentals and RTSP streaming

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 3, 2026

- ▶ Days 1 to 10 ran a model on one device. Week 3 connects the devices: a camera node sends images, a second node runs the model, and a third node shows the results.
- ▶ A network adds delay, loses packets, and has a limited bandwidth. A video stream at 30 frames each second can need more bandwidth than the Wi-Fi of a classroom gives to one device.
- ▶ A detector on a network stream sees an image that is old. The age of the image comes from the camera, the encoder, the network, the buffers, and the model. The latency of the model is only one part.
- ▶ In the lab you find the devices on the lab network, publish the camera of the Raspberry Pi as an RTSP stream, run the Day 8 detector on the stream, and measure the end-to-end latency. Then you stream the camera of the XIAO and compare.

After this day you can:

- ▶ Explain when to use TCP and when to use UDP.
- ▶ Describe an RTSP session and the role of RTP and H.264.
- ▶ Set up an RTSP server on the Raspberry Pi.
- ▶ Read a video stream in Python and run inference on it.
- ▶ Measure the end-to-end latency of a stream.

Part 1 Networking for edge devices

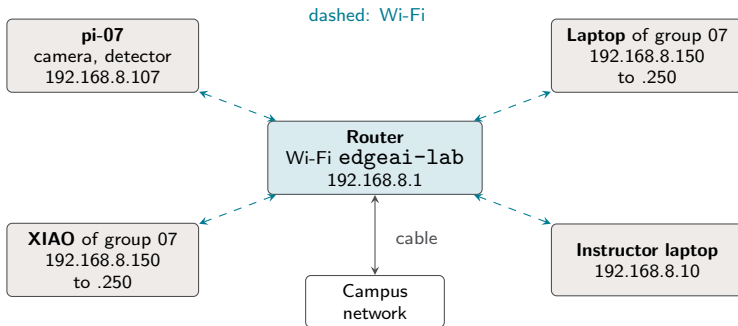
Exercise: read the output of three network commands and find the fault

Part 2 Video streaming

Exercise: calculate the bit rate of an MJPEG and an H.264 stream

Part 3 Stream architectures

Exercise: select the inference location for three camera systems

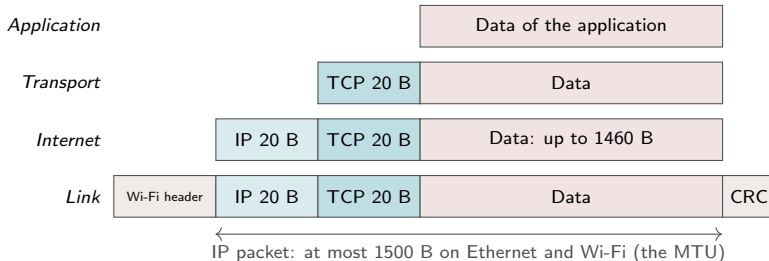


- ▶ Days 11 to 13 send data from one device to another: images, messages, measurements.
- ▶ One router for the lab. A campus network can block the traffic between two devices.
- ▶ The Raspberry Pi of group NN has the fixed address X. (100+NN), here with $X = 192.168.8$. The other devices get an address from a pool.

OSI model	Internet model	Examples in this course
7 Application	Application	HTTP, RTSP, RTP, MQTT, SSH, DNS, DHCP
6 Presentation		
5 Session	Transport	TCP, UDP
4 Transport		
3 Network	Internet	IP, ICMP (ping)
2 Data link	Link	Wi-Fi (IEEE 802.11), Ethernet (IEEE 802.3)
1 Physical		

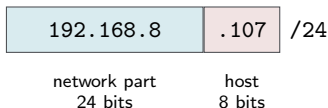
- ▶ Each layer uses the service of the layer below it. It does not know how that layer works.
- ▶ The internet uses four layers. The OSI model has seven and gives the names that you read in documentation: “layer 2”, “layer 7”.
- ▶ A program selects only the application protocol and the transport. The operating system does the rest.
- ▶ Search for a fault from the bottom to the top: link, address, reach, port, application.

Each layer adds a header



- ▶ The headers have a fixed minimum size: IP 20 bytes, TCP 20 bytes, UDP 8 bytes.
- ▶ The MTU limits one IP packet to 1500 bytes. So one TCP segment carries at most $1500 - 20 - 20 = 1460$ bytes of data.
- ▶ A JPEG image of 30 KB (30 720 bytes) needs $\lceil 30\,720/1460 \rceil = 22$ packets. If one packet is lost, the image is not complete.
- ▶ The IP and TCP headers are $40/1500 = 2.7$ percent of a full packet. The headers of many small messages cost much more.

Source: RFC 791, 9293, 768, and 894. The image is an example.

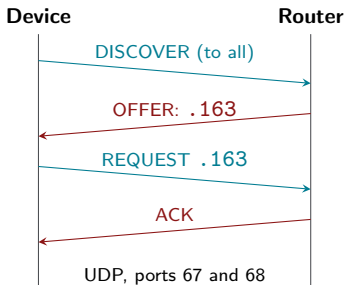


Address	Meaning
192.168.8.0	The network
192.168.8.1	The router (gateway)
.2 to .254	253 devices
192.168.8.255	Broadcast: all devices

- ▶ An IPv4 address has 32 bits. /24 says that the first 24 bits name the network.
- ▶ Two devices in the same network send to each other directly. To another network they send through the router.
- ▶ Private networks: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. The router translates them to its one public address.
- ▶ 127.0.0.1: this computer (loopback).
- ▶ 169.254.x.x: the device got no answer from a DHCP server.

Source: RFC 1918 (private addresses), RFC 3927 (169.254.0.0/16).

DHCP: a device gets its address

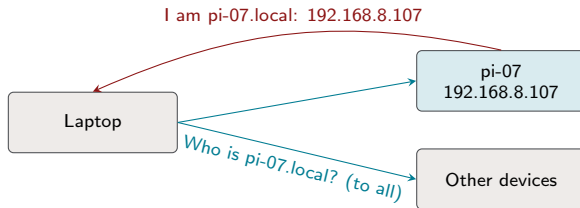


The ACK gives the device:

- ▶ its address and the mask (/24),
- ▶ the gateway (the router),
- ▶ the DNS server,
- ▶ the lease time: the address is valid for this time.
- ▶ A device has no address before DHCP, so it sends the first message to all devices.
- ▶ A reservation in the router gives one device always the same address. The router knows the device by its MAC address.

Key point: The lab reserves 192.168.8.107 for pi-07. Your code can then use this address for the full course.

Source: RFC 2131 (DHCP). The address .163 is an example.



- ▶ A program needs an address. A person prefers a name.
- ▶ **DNS**: a server answers “which address has this name?”. The router or the internet provider runs it (UDP port 53).
- ▶ **Multicast DNS**: a name that ends with `.local`. The laptop asks all devices of the network. The device with the name answers. No server is necessary (UDP port 5353).
- ▶ Raspberry Pi OS answers for its host name with the program `avahi`. So `ssh edge@pi-07.local` works.
- ▶ Some networks block multicast. Then use the address.

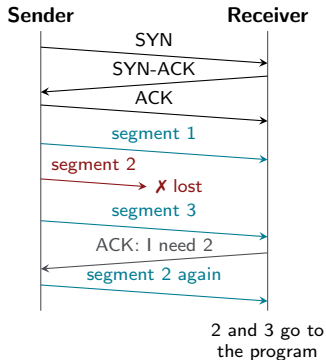
Source: RFC 1035 (DNS), RFC 6762 (multicast DNS).

One device runs many programs. The port number selects the program:
192.168.8.107:8554 is the RTSP server on pi-07.

Port	Transport	Service in this course
22	TCP	SSH to the Raspberry Pi
53, 5353	UDP	DNS and multicast DNS
67, 68	UDP	DHCP
80	TCP	HTTP: the MJPEG stream of the XIAO (Day 11)
1883	TCP	MQTT broker (Days 12 and 13)
5000	TCP	Web page of the Day 8 detector
5201	TCP, UDP	iperf3
8554	TCP	RTSP server MediaMTX (Day 11)
11434	TCP	Ollama (Day 10)

- ▶ A server *listens* on a fixed port. A client uses a free port that the system selects: on Linux from 32 768 to 60 999.
- ▶ A port below 1024 needs administrator rights on Linux. So the course servers use 1883, 5000, and 8554.

TCP: a reliable byte stream



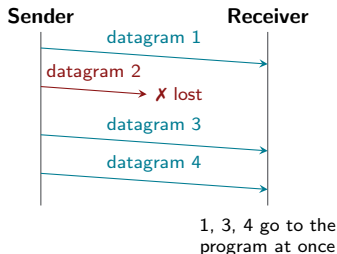
- ▶ A connection starts with three messages. The first data leaves after one round trip.
- ▶ The receiver confirms the data (ACK). The sender sends a lost segment again.
- ▶ The program receives all bytes, in the correct order.
- ▶ TCP also adapts its rate to the network (congestion control).

The cost: segment 3 waits for segment 2. One lost packet delays all data after it.

When no ACK comes, Linux waits at least 200 ms before it sends the segment again. At 30 frames each second, this is 6 frames.

Source: RFC 9293 (TCP). Linux kernel 5.15, `include/net/tcp.h`:
`TCP_RTO_MIN` is `HZ/5`, 200 ms.

UDP: single datagrams with no resend



- ▶ No connection: the first datagram leaves at once.
 - ▶ No ACK, no resend, no order. A header of 8 bytes.
 - ▶ Each datagram arrives complete, or it does not arrive.
 - ▶ The program decides what to do with a loss: skip it, hide it, or ask again.
-
- ▶ A video packet that arrives 200 ms late is too late for the display. With UDP the stream continues; the image has an error for a short time.
 - ▶ UDP has no rate control. A sender can fill the network and lose its own packets.

Source: RFC 768 (UDP).

Protocol	Transport	Reason
SSH, HTTP, MQTT	TCP	A command or a message must arrive complete
MJPEG over HTTP	TCP	Simple; each image arrives complete or late
RTSP (the control)	TCP	Few messages, all necessary
RTP (the video)	UDP, or TCP	An old image has no value
DNS, DHCP	UDP	One short question, one short answer

Ask two questions:

1. Has the data a value when it arrives late? Video, audio, and a live sensor value: no. A file, a command, an alarm: yes.
 2. Can the program accept a gap? A stream of measurements: yes. A model file or a configuration: no.
- A firewall or a router can block UDP. Then a stream uses TCP: RTSP can carry the RTP packets in its TCP connection.

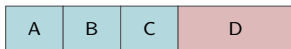
Key point: Data that must arrive complete: TCP. Data that is useless when late: UDP.

	Ethernet (cable)	Wi-Fi (radio)
Medium	One cable for each device. Send and receive at the same time.	One radio channel for all devices. One sender at a time.
Delay	Low and stable	Changes with the distance, the walls, and the other devices
Loss	Very rare	Frequent. The radio sends a lost frame again, and this adds delay.
Raspberry Pi 5	Gigabit: 1 Gb/s	802.11ac, 2.4 and 5 GHz: 300 Mb/s
XIAO ESP32S3	None	802.11 b/g/n, 2.4 GHz only: up to 150 Mbit/s

- ▶ The Wi-Fi values are link rates of the radio. A program gets much less: measure the throughput with `iperf3`.
- ▶ If the lab has a switch, connect the Raspberry Pi with a cable. Its video stream then does not use the Wi-Fi channel twice.

Source: Raspberry Pi documentation (CC BY-SA 4.0).
ESP32-S3 data sheet, version 2.2.

Time of the channel



A, B, C: near the router, fast.

D: far away, slow. The same data needs more time.

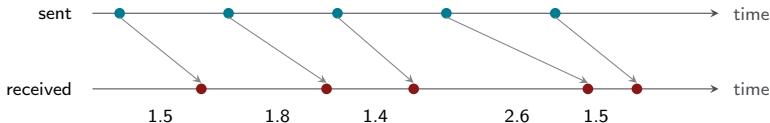
- ▶ All devices on one channel share its time. When one device sends, the others wait.
- ▶ A device far from the router uses a slow rate. Its data needs more time, and all devices get less.
- ▶ The 2.4 GHz band has 3 channels that do not overlap: 1, 6, and 11.

Time for 1 MB on 802.11n, one stream, 20 MHz:

Rate of the radio	Link rate	Time for 1 MB
Near the router (MCS 7, short guard interval)	72.2 Mbit/s	0.116 s
Far from the router (MCS 0)	6.5 Mbit/s	1.291 s

Source: Link rates of IEEE 802.11n. Slow device: Heusse et al., INFOCOM 2003. The times ignore the headers and the waits.

Bandwidth, latency, jitter, and loss



Delay of five packets sent at a regular interval (example values in units of time).

Value	Meaning	Tool
Bandwidth	The capacity of a link, in bits each second	Data sheet
Throughput	The rate that a program really gets	iperf3
Latency	The time of one packet. <code>ping</code> gives the round trip (RTT).	<code>ping</code>
Jitter	The change of the latency from packet to packet	<code>ping (mdev)</code> , <code>iperf3 -u</code>
Loss	The part of the packets that never arrive	<code>ping</code> , <code>iperf3 -u</code>

A network rate uses powers of 10: 1 Mbit/s is 10^6 bits each second. A file has bytes: multiply by 8.

The four parts of the delay

One packet on one link has four delays:

Delay	Formula	Example
Transmission	bits / link rate	A JPEG of 30 KB at 72.2 Mbit/s: 3.40 ms. At 1 Gbit/s: 0.25 ms.
Propagation	distance / speed of the signal	20 m of radio in the room: 0.07 μ s. 3600 km of fibre: 18 ms.
Queueing	the wait behind other packets	0 on an empty network. Large on a full one.
Processing	the work of each device	Small for a router. Large for an encoder or a model.

- ▶ In a room, the propagation is almost zero. The transmission and the queues set the delay.
- ▶ Over a long distance, the propagation sets a floor: round trip $\geq 2 \times$ distance / 200 000 km/s in fibre. 3600 km give 36 ms before any computation.
- ▶ A full queue in the router adds delay to every packet. A stream that sends more than the link carries fills it.

Source: Light barrier: *Machine Learning Systems*, Volume I, chapter “ML Systems”.

The image size and the distance are examples.

ping, 100 requests, one each 0.2 s, from the work computer (connected by cable):

Target	min	avg	max	mdev
127.0.0.1: this computer	0.034	0.069	0.105	0.014
1.1.1.1: a public DNS server	69.353	69.658	69.927	0.129
8.8.8.8: a second DNS server	69.197	69.453	69.637	0.114

Round-trip times in ms. 0 of 300 requests were lost.

- ▶ The loopback has no network: 0.07 ms is the time of the software.
- ▶ The two servers are 69 ms away. With the light barrier, the fibre path is at most $34.6 \text{ ms} \times 200 \text{ km/ms} = 6920 \text{ km}$ long.
- ▶ The cable path has a small jitter: mdev 0.11 to 0.13 ms. Wi-Fi has more. You measure it in the lab.
- ▶ A detector with a budget of 50 ms cannot wait for a server at 69 ms. The edge device must decide alone.

Source: Experiment of this course: ping of iputils 20211215 on the x86 work computer. The values say nothing about the lab network.

```
$ ip -br addr
lo                UNKNOWN          127.0.0.1/8 ::1/128
eth0              DOWN
wlan0             UP              192.168.8.107/24 fe80::5a1c:2e9f:7d40:b3c1/64
$ ip route
default via 192.168.8.1 dev wlan0 proto dhcp src 192.168.8.107 metric 600
192.168.8.0/24 dev wlan0 proto kernel scope link src 192.168.8.107 metric 600
$ ip neigh
192.168.8.1 dev wlan0 lladdr 02:11:22:33:44:01 REACHABLE
192.168.8.163 dev wlan0 lladdr 02:11:22:33:44:a3 STALE
```

- ▶ `ip -br addr`: each interface, its state, and its address. Here the Wi-Fi wlan0 has 192.168.8.107/24, and the cable eth0 is not connected.
- ▶ `ip route`: default via names the router. The second line says that the network 192.168.8.0/24 is directly on wlan0.
- ▶ `ip neigh`: the devices that this computer reached recently, with their MAC addresses.

Source: Example output of pi-07 in the format of iproute2. Nobody measured these values.

```
$ ss -tln
State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0      128          0.0.0.0:22      0.0.0.0:*
LISTEN 0      4096         *:8554          *:*
LISTEN 0        5        127.0.0.1:5000  0.0.0.0:*
$ ss -tn
State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
ESTAB  0        0    192.168.8.107:22  192.168.8.163:51022
ESTAB  0    412360  192.168.8.107:8554  192.168.8.163:51388
```

- Options: `-t` TCP, `-u` UDP, `-l` listening only, `-n` numbers, `-p` the program.
- `0.0.0.0` or `*`: the server accepts connections on all interfaces.
`127.0.0.1`: only from this computer. Port 5000 here does not answer the laptop.
- Send-Q of a connection: the bytes that the peer did not confirm yet. A large value that grows means: the network or the reader is too slow for the stream.

Source: Example output of `ss -tln` in the format of `ss -tln`. Nobody measured these values.

```
$ ping -c 4 pi-07.local
PING pi-07.local (192.168.8.107) 56(84) bytes of data.
64 bytes from pi-07.local (192.168.8.107): icmp_seq=1 ttl=64 time=3.84 ms
64 bytes from pi-07.local (192.168.8.107): icmp_seq=2 ttl=64 time=2.91 ms
64 bytes from pi-07.local (192.168.8.107): icmp_seq=4 ttl=64 time=41.2 ms

--- pi-07.local ping statistics ---
4 packets transmitted, 3 received, 25% packet loss, time 3005ms
rtt min/avg/max/mdev = 2.910/15.983/41.200/17.835 ms
```

- ▶ The name resolves and the device answers. Request 3 was lost, and request 4 needed 41.2 ms: a sign of a weak Wi-Fi signal or a full channel.
- ▶ Use 100 requests or more for a jitter value. Four are too few.

Message	Meaning
Name or service not known	The name does not resolve: use the address
Destination Host Unreachable	No device answers at this address in your network
100% loss, no message	The device is off, or it blocks ping

Source: Example output in the format of `iputils`. Nobody measured these values.

```
pi-07$ iperf3 -s                                # server, port 5201
laptop$ iperf3 -c pi-07.local -t 10             # TCP, laptop to pi-07
laptop$ iperf3 -c pi-07.local -t 10 -R          # TCP, pi-07 to laptop
laptop$ iperf3 -c pi-07.local -u -b 20M         # UDP at 20 Mbit/s
```

- ▶ `-R` reverses the direction. The video goes from the Raspberry Pi to the laptop: measure this direction.
- ▶ With `-u` the client sends UDP at the rate `-b`. The last lines give the jitter and the lost datagrams.

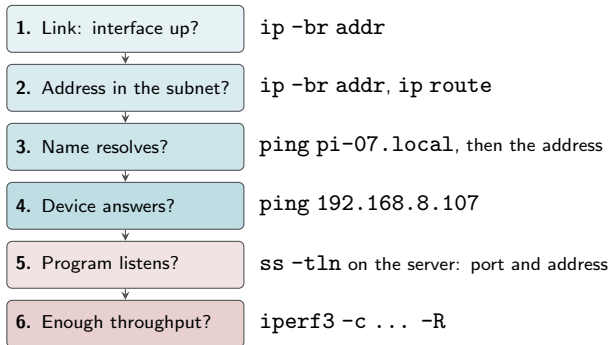
Measured on one computer, through the loopback:

[ID]	Interval		Transfer	Bitrate	Retr	
[5]	0.00-5.00	sec	...	19.2 Gbits/sec	0	sender
[ID]	Interval		Transfer	Bitrate	Jitter	Lost/Total Datagrams
[5]	0.00-5.04	sec	...	20.0 Mbits/sec	0.011 ms	0/382 (0%) receiver

- ▶ 19.2 Gbit/s is the speed of the memory, not of a network. A test needs two devices.

Source: Experiment of this course: iperf 3.9 on the x86 work computer, 5 s each.
The interval and the transfer columns are not shown.

A method to find a network fault



- ▶ Go from the bottom layer to the top layer. Stop at the first step that fails: the fault is there.
- ▶ Test from the device that has the problem. A laptop that reaches pi-07 says nothing about the XIAO.

Exercise: find the fault

Group 07 cannot open `rtsp://pi-07.local:8554/cam` on the laptop. The video player shows an error. The group runs three commands:

```
laptop$ ip -br addr
lo                UNKNOWN          127.0.0.1/8 ::1/128
wlan0             UP              192.168.8.163/24 fe80::9c2d:41ff:fe6a:20d7/64

laptop$ ping -c 3 pi-07.local
64 bytes from pi-07.local (192.168.8.107): icmp_seq=1 ttl=64 time=4.12 ms
64 bytes from pi-07.local (192.168.8.107): icmp_seq=2 ttl=64 time=2.87 ms
64 bytes from pi-07.local (192.168.8.107): icmp_seq=3 ttl=64 time=3.30 ms
3 packets transmitted, 3 received, 0% packet loss, time 2003ms

pi-07$ ss -tln
State  Recv-Q  Send-Q  Local Address:Port  Peer Address:Port  Process
LISTEN 0        128          0.0.0.0:22          0.0.0.0:*
LISTEN 0       4096      127.0.0.1:8554      0.0.0.0:*
```

For each output, say which step of the method passes. Then find the fault and give the change that corrects it. Work with your neighbour for 6 minutes.

Source: Example outputs. Nobody measured these values.

1. `ip -br addr`: the Wi-Fi is up, and the laptop has 192.168.8.163/24 from DHCP. Steps 1 and 2 pass.
2. `ping`: the name resolves to 192.168.8.107, and pi-07 answers with no loss. Steps 3 and 4 pass.
3. `ss -tln`: the RTSP server listens on 127.0.0.1:8554. Only programs on pi-07 can connect. Step 5 fails.

Correction: set `rtspAddress`: :8554 in the configuration of MediaMTX and start the server again. `ss -tln` then shows *:8554.

Other output	Fault
169.254.x.x in <code>ip -br addr</code>	No DHCP answer
Destination Host Unreachable	pi-07 is off, or in another network
No line with :8554 in <code>ss</code>	The server does not run

Key point: A device that answers `ping` can still refuse a service. Check the port and its address on the server.

Part 1 Networking for edge devices

Exercise: read the output of three network commands and find the fault

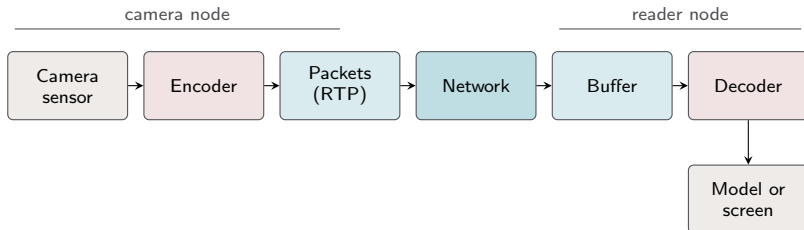
Part 2 Video streaming

Exercise: calculate the bit rate of an MJPEG and an H.264 stream

Part 3 Stream architectures

Exercise: select the inference location for three camera systems

From the camera to the model



- ▶ A camera makes a frame each 33 ms at 30 frames each second. The network cannot carry the raw frames.
- ▶ The encoder makes the frames small. The protocol cuts them into packets and gives each packet a time.
- ▶ The reader collects the packets, waits for missing ones, and decodes the frame for the model.
- ▶ Each step adds delay. Part 3 measures the sum.

Key point: This part: how large a stream is, how an encoder makes it small, and how RTSP, RTP, and RTCP carry it.

- ▶ **Resolution:** the pixels of one frame, width $\times$ height.
- ▶ **Frame rate:** the frames each second (fps).
- ▶ **Bit rate:** the bits each second of the stream, in Mbit/s.

Raw video at 30 frames each second:

Resolution	RGB frame	RGB stream	YUV 4:2:0 frame	YUV 4:2:0 stream
640×360	675 KB	165.9 Mbit/s	338 KB	82.9 Mbit/s
1280×720	2700 KB	663.6 Mbit/s	1350 KB	331.8 Mbit/s
1920×1080	6075 KB	1493.0 Mbit/s	3038 KB	746.5 Mbit/s

- ▶ Bit rate = bytes of one frame $\times 8 \times$ frames each second. $1280 \times 720 \times 3 \times 8 \times 30 = 663.6$ Mbit/s.
- ▶ The XIAO has a Wi-Fi link of at most 150 Mbit/s. One Wi-Fi channel serves all groups.
- ▶ A stream for the network must be 100 times smaller or more. The encoder does this.



4 brightness values

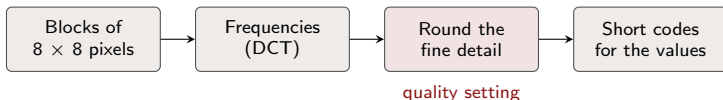


2 colour values
for the 4 pixels

4 pixels: $4 + 1 + 1 = 6$ bytes, so 1.5 bytes for each pixel. RGB needs 3.

- ▶ Y is the brightness. U and V are the colour.
- ▶ The eye sees fine detail in the brightness, not in the colour. So one U and one V serve a square of 4 pixels.
- ▶ Video encoders work on YUV 4:2:0. This halves the raw size before the compression starts.
- ▶ A model needs RGB. The reader converts each decoded frame. This costs time on the board.

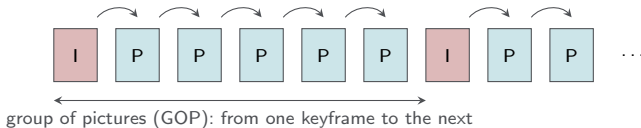
MJPEG: one JPEG image for each frame



- ▶ Motion JPEG compresses each frame alone, as a photo. The stream is a series of JPEG files.
- ▶ A lost frame damages only that frame. Each frame is a start point, so a new reader shows an image at once.
- ▶ The coding needs little computation. The camera module of the XIAO makes the JPEG itself: the stream sketch reads PIXFORMAT_JPEG frames.
- ▶ The quality setting selects how much detail the rounding removes. On the XIAO camera, `jpeg_quality` goes from 0 to 63, and a lower value gives a better image and a larger file. The sketch uses 12.
- ▶ The cost: the stream sends the same background again in each frame.

Source: Stream sketch `Streaming_Video.ino` of the repository `XIAO-ESP32S3-Sense`.

H.264: send the changes



- ▶ **I-frame** (keyframe): coded alone, like a JPEG. A decoder can start here.
- ▶ **P-frame**: for each block, a motion vector to a similar block of the frame before, and the small difference.
- ▶ **B-frame**: also uses a later frame. The encoder must wait for that frame, so B-frames add delay. A live stream uses none: the option `zerolatency` of `x264` and `--low-latency` of `rpicam-vid`.
- ▶ A lost packet damages its frame and all later frames that copy from it, until the next keyframe.

Source: Raspberry Pi documentation, "Camera software", section "Streaming" (CC BY-SA 4.0).

One test video, 1280×720 , 30 frames each second, 10 s:

Format	Moving camera		Fixed camera	
	Mbit/s	PSNR	Mbit/s	PSNR
Raw YUV 4:2:0	331.8		331.8	
MJPEG, quality 5	11.09	42.0 dB	10.23	41.3 dB
H.264, keyframe 1 s	2.62	41.3 dB	0.97	40.5 dB

PSNR: the similarity to the original frames; a higher value is better. Settings: -q:v 5; x264 veryfast, zerolatency, CRF 23.

- ▶ The two formats give almost the same image quality.
- ▶ Moving camera: H.264 needs 4.2 times fewer bits than MJPEG. Fixed camera: 10.5 times fewer. The fixed scene changes only by noise.
- ▶ MJPEG has almost the same bit rate for the two scenes. It does not use the frames before.
- ▶ Compression against raw YUV: 30 to 32 times for MJPEG, 127 to 341 times for H.264.

Source: Experiment of this course: FFmpeg 4.4, x86 work computer.
Video of the repository XIAO-ESP32S3-Sense (Apache-2.0).

Mean size of a frame, 1280×720 , keyframe each 30 frames:

Scene	I-frame	P-frame	I / P
Moving camera	38.4 KB	9.7 KB	4.0
Fixed camera	78.8 KB	1.4 KB	56.9

- ▶ A fixed camera gives almost empty P-frames. Most of its bits are in the keyframes.
- ▶ The encoder gives a keyframe more bits when the next frames copy from it. With only keyframes, a frame of the fixed scene has 19.8 KB.
- ▶ A keyframe is a burst. At 2 Mbit/s, the keyframe of the fixed scene (64.1 KB) needs 0.26 s on the link: almost 8 frame times.
- ▶ The network sees a peak each second. A link with only the mean bit rate delays each keyframe.

Source: Experiment of this course: FFmpeg 4.4, x86 work computer.
 $0.26 \text{ s} = 65\,631 \text{ bytes} \times 8 / 2 \text{ Mbit/s}$.

Keyframe	Each frame	Each 1 s	Each 10 s
Moving camera, Mbit/s	5.27	2.62	2.46
Fixed camera, Mbit/s	4.86	0.97	0.35
Wait of a new reader for the first keyframe (median of 10)	0	0.50 s	5.51 s

1280 × 720, x264 CRF 23. The wait: 10 readers of an RTSP stream at 2 Mbit/s, each started at a random time.

- ▶ A short interval costs bits. A long one saves bits, mostly for a fixed camera.
- ▶ A new reader can decode only from a keyframe. With a keyframe each 10 s, it waits up to 10 s for the first image (here at most 7.40 s).
- ▶ After a lost packet, the image has errors until the next keyframe.
- ▶ The lab uses one keyframe each second (`rpiCameraIDRPeriod: 30`).

Source: Experiment of this course: FFmpeg 4.4 and MediaMTX v1.21.1 on the x86 work computer.

- ▶ **Constant quality** (x264: CRF): each frame gets the bits that it needs. The bit rate follows the scene.
- ▶ **Bit rate limit**: the encoder keeps the stream at a set value and changes the quality. A network can plan with it.

Measured, 1280×720 , keyframe each 1 s:

Scene	CRF 23		Limit 2 Mbit/s	
	Mbit/s	PSNR	Mbit/s	PSNR
Moving camera	2.62	41.3 dB	2.01	40.7 dB
Fixed camera	0.97	40.5 dB	2.08	40.6 dB

- ▶ Moving camera: the limit removes 23 percent of the bits and 0.7 dB of quality.
- ▶ Fixed camera: the limit sends 2.1 times the bits for the same quality.
- ▶ The lab starts with 2 Mbit/s (`rpiCameraBitrate`). Part C of the lab changes it.

Source: Experiment of this course: FFmpeg 4.4 on the x86 work computer.

The encoders of the two boards

	XIAO ESP32S3 Sense	Raspberry Pi 5
Formats	JPEG from the camera module	H.264 and MJPEG in software
Encoder	In the camera module: no CPU time	On the four CPU cores
Stream in the lab	MJPEG over HTTP, port 80	H.264 over RTSP, port 8554 (MediaMTX)
Effect	A large stream	Less CPU for the detector

- ▶ “Raspberry Pi 5 uses software video encoders.” The Raspberry Pi 4 had a hardware H.264 encoder.
- ▶ MediaMTX selects the software encoder on the Raspberry Pi 5 (`rpiCameraCodec: auto`).
- ▶ MediaMTX can add a second stream of the same camera with its own size and rate. Its default codec is MJPEG.

Key point: On the Raspberry Pi 5, the encoder and the detector share the CPU. Measure the detector with the stream active.

Source: Raspberry Pi documentation, “`rpivid`” (CC BY-SA 4.0).
MediaMTX documentation, “Raspberry Pi Cameras” (MIT).

RTP: packets with a time stamp

V	P	X	CC	M	PT	sequence number	4 bytes
timestamp (32 bits)							4 bytes
SSRC: the source of the stream (32 bits)							4 bytes
payload: a part of an H.264 frame							

- ▶ A header of 12 bytes. RTP runs on UDP, or inside a TCP connection.
- ▶ **Sequence number**: plus 1 for each packet. The reader finds lost packets and puts packets in order.
- ▶ **Timestamp**: the capture time of the frame. Video uses a clock of 90 000 steps each second: 3000 steps for each frame at 30 frames each second.
- ▶ **M** (marker): the last packet of a frame. **PT**: the payload type; the session description gives its codec.

Source: RFC 3550 (RTP), RFC 6184 (H.264 in RTP).

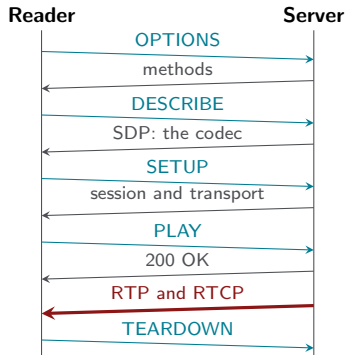
Measured: the RTP packets of one stream

seq	timestamp	M	bytes	
2582	2458257548	0	1452	
2583	2458257548	0	1452	
2584	2458257548	0	1452	
2585	2458257548	0	1452	
2586	2458257548	1	763	<- last packet of the frame
2587	2458260548	0	1452	<- next frame: timestamp + 3000

2 s of the stream (1280×720 , 2 Mbit/s, keyframe each 1 s):

- ▶ 461 packets for 61 frames: 7.6 packets for each frame, 29 for a keyframe.
- ▶ The largest packet has 1452 bytes. A frame that is larger becomes several packets, so it fits the MTU of 1500 bytes.
- ▶ One lost packet out of 7.6 makes the frame incomplete.
- ▶ The timestamp step is 3000 for each frame: $90\,000/30$.

Source: Experiment of this course: MediaMTX v1.21.1, x86 work computer.
The table shows fields of the packets, not the output of a tool.



- ▶ A text protocol like HTTP, on one TCP connection. Port 554 by default; MediaMTX uses 8554.
- ▶ DESCRIBE returns the session description (SDP): the codec, the clock, the tracks.
- ▶ SETUP selects the transport: RTP on UDP, or inside this TCP connection.
- ▶ PLAY starts the stream. TEARDOWN ends it.
- ▶ Cameras and MediaMTX use RTSP 1.0 (RFC 2326). RFC 7826 is version 2.0.

Source: RFC 2326 (RTSP 1.0), RFC 7826 (RTSP 2.0), RFC 8866 (SDP).

```
DESCRIBE rtsp://127.0.0.1:18554/cam RTSP/1.0
Accept: application/sdp

RTSP/1.0 200 OK
Content-Type: application/sdp
...
m=video 0 RTP/AVP 96
a=control:trackID=0
a=rtpmap:96 H264/90000
a=fmtp:96 packetization-mode=1; profile-level-id=64001F; sprop-parameter-...

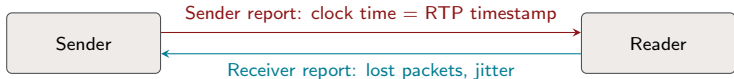
SETUP rtsp://127.0.0.1:18554/cam/trackID=0 RTSP/1.0
Transport: RTP/AVP/TCP;unicast;interleaved=0-1

RTSP/1.0 200 OK
Session: 169fbdffad7546aa9b4aabd6696cab66;timeout=55
Transport: RTP/AVP/TCP;unicast;interleaved=0-1;ssrc=9E9DC1FB
```

- ▶ rtpmap:96 H264/90000: payload type 96 is H.264 with the clock of 90 000 steps each second.
- ▶ interleaved=0-1: RTP on channel 0, RTCP on channel 1.

Source: Experiment of this course: MediaMTX v1.21.1. Lines removed (...).

RTCP: reports about the stream



- ▶ RTCP runs next to RTP: on the next UDP port, or on the next channel of the RTSP connection.
- ▶ The **sender report** links the RTP timestamp to the clock time of the sender. The reader can then find the capture time of a frame and synchronize audio and video.
- ▶ The **receiver report** gives the part of the packets that were lost and the jitter. A sender can lower its bit rate.
- ▶ RTCP uses at most 5 percent of the bandwidth of the session.
- ▶ In the experiment, the first sender report came 0.08 s after PLAY: 25 packets, 33 204 bytes sent.

Source: RFC 3550, sections 6.2 and 6.4. Experiment of this course: MediaMTX v1.21.1.

	UDP	TCP (interleaved)
Ports	Two more ports: 8000 (RTP) and 8001 (RTCP) on the server	The RTSP connection only
Loss	A lost packet stays lost: the image has errors until the next keyframe	TCP sends the packet again: no image errors
Delay	Low and stable	Grows on a bad link: later packets wait for the resend
Network	A firewall or a router can block it	Passes where the RTSP connection passes

- ▶ The MediaMTX documentation: UDP is “the most performant”, TCP “the most versatile”.
- ▶ MediaMTX drops packets for a reader that is too slow. A larger write queue keeps more packets and uses more RAM.
- ▶ The lab uses TCP first. Part C of the lab tries UDP and compares the delay.

Source: MediaMTX documentation (MIT): “RTSP-specific features”, “Decrease packet loss”.

```
HTTP/1.1 200 OK
Content-Type: multipart/x-mixed-replace;boundary=123456789000000000000987654321

--12345678900000000000000987654321
Content-Type: image/jpeg
Content-Length: 23816

<JPEG data of frame 1>
--12345678900000000000000987654321
Content-Type: image/jpeg
...
```

- ▶ The XIAO answers one HTTP request with a stream of JPEG parts that never ends. A browser shows each new part in place of the last.
- ▶ No RTP, no time stamp, no special reader: a browser, OpenCV, or VLC opens `http://<address>/`.
- ▶ One TCP connection for each viewer. Each viewer costs the full bit rate on the Wi-Fi of the XIAO.

Source: Stream sketch `Streaming_Video.ino` of XIAO-ESP32S3-Sense.
The value of `Content-Length` is an example.

Compare the ways to stream

Method	Transport	Good for	Limit
MJPEG over HTTP	TCP	A microcontroller, a browser, a simple reader	A large stream
RTSP with RTP	TCP or UDP	Cameras, a program that reads the stream	No direct view in a browser
HLS	HTTP files	Many viewers, the internet	Delay of a few segments of seconds
WebRTC	UDP	A browser with a short delay	A complex setup

- ▶ MediaMTX reads and serves RTSP, HLS, WebRTC, and other protocols. It converts a stream from one protocol to another.
- ▶ HLS cuts the stream into files. A reader can show a file only when the file is complete, so the delay is at least one file.
- ▶ The lab: MJPEG over HTTP from the XIAO, and RTSP from the Raspberry Pi to a Python reader.

Source: MediaMTX README and documentation, "Read streams" (MIT).

Two streams of one group. The sizes are example values, near the values of the experiment.

1. The XIAO sends MJPEG: 640×480 , 10 frames each second, a mean JPEG of 24 KB.
2. The Raspberry Pi sends H.264: 1280×720 , 30 frames each second, one keyframe each second. A keyframe has 40 KB, a P-frame 9 KB.

Calculate:

- a. the bit rate of each stream in Mbit/s (1 KB = 1024 bytes);
- b. the compression factor of the H.264 stream against raw YUV 4:2:0 (331.8 Mbit/s);
- c. the total of 12 groups. Does it fit a Wi-Fi channel that carries 30 Mbit/s?
- d. the time of one keyframe on a link of 2.4 Mbit/s.

Work with your neighbour for 8 minutes.

- a. MJPEG: $24 \times 1024 \times 8 \times 10 = 1.97 \text{ Mbit/s}$.
H.264: $(40 + 29 \times 9) \times 1024 \times 8 = 301 \times 8192 = 2.47 \text{ Mbit/s}$.
- b. $331.8/2.47 = 135$. The MJPEG stream has a third of the frames and a smaller image, and it still needs 80 percent of the H.264 bit rate.
- c. One group: $1.97 + 2.47 = 4.43 \text{ Mbit/s}$. 12 groups: 53.2 Mbit/s . It does not fit. Options: fewer frames from the XIAO, a lower bit rate, a cable for each Raspberry Pi, or the 5 GHz band for the Raspberry Pi.
- d. $40 \times 1024 \times 8/2400000 = 0.137 \text{ s}$: about 4 frame times. Each keyframe adds this delay.

Key point: Bit rate = bytes of a frame $\times 8 \times$ frames each second. For H.264, add the keyframes and the P-frames of one GOP.

Part 1 Networking for edge devices

Exercise: read the output of three network commands and find the fault

Part 2 Video streaming

Exercise: calculate the bit rate of an MJPEG and an H.264 stream

Part 3 Stream architectures

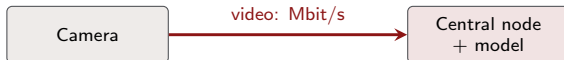
Exercise: select the inference location for three camera systems

Where does the model run?

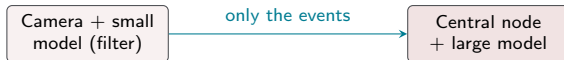
A. Camera node



B. Central node



C. Split



- ▶ A: the camera node runs the model and sends results. Example: the Raspberry Pi with its camera runs the Day 8 detector.
- ▶ B: the camera sends video. A central node runs the model for many cameras. Example: the XIAO sends MJPEG to the Raspberry Pi.
- ▶ C: a small model on the camera node selects the frames that the large model sees.

The textbook compares two complete paths with the deadline of the service:

$$T_{\text{remote}} = \frac{D}{\text{BW}} + L_{\text{network}} + \frac{O}{R_{\text{remote}} \cdot \eta} \quad \text{remote is possible only if } T_{\text{remote}} \leq T_{\text{deadline}}$$

Symbol	Meaning
D	The data of one request: a frame, or a part of a stream
BW	The bandwidth to the remote node
L_{network}	The round trip of the network (Part 1)
$O, R \cdot \eta$	The operations of the model, and the real rate of the remote processor

- ▶ Compute T_{local} in the same way: the model time on the camera node. Then compare the paths that meet the deadline.
- ▶ The result is small: a class or a few boxes. Its transfer time is almost zero.
- ▶ A faster remote processor does not help when the transfer is the largest part.

Source: *Machine Learning Systems*, Volume I, chapter “ML Systems”, section “The data locality invariant”.

12 cameras on one Wi-Fi channel:

What the camera node sends	One camera	12 cameras
H.264, 1280 × 720, keyframe each 1 s (Part 2)	2.62 Mbit/s	31.4 Mbit/s
MJPEG, 1280 × 720, quality 5 (Part 2)	11.09 Mbit/s	133.1 Mbit/s
Results: 10 frames each second, 300 bytes of JSON each	0.024 Mbit/s	0.29 Mbit/s

- ▶ The results need about 100 times fewer bits than the H.264 stream.
- ▶ 12 MJPEG streams do not fit one Wi-Fi channel. 12 H.264 streams use most of it, and the channel then has no room for other traffic.
- ▶ With the results only, the channel stays almost free, and a device with a weak signal does not slow the others.

Key point: Each camera that sends video uses the shared channel. A camera that sends results uses almost nothing.

Source: Bit rates: Part 2 (moving camera). The size of the JSON message is an example.

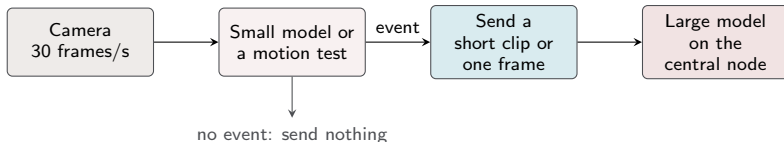
Compare the two places

	A. Model on the camera node	B. Model on a central node
Network	Results only	A video stream for each camera
Delay	No video path: capture, model	Encoder, network, buffers, decoder, model
Model size	Limited by the camera node	A larger model, a faster processor
Without network	Still decides	Stops
Updates	One update for each camera	One update for all cameras
Cost and power	A processor in each camera	Simple cameras, one strong node
Privacy	Images can stay on the camera	Images leave the camera

- ▶ No place wins in all rows. The deadline, the network, and the number of cameras decide.
- ▶ The textbook: local processing “reduces the data attack surface”. Data that stays on the device still needs access control.

Source: *Machine Learning Systems*, Volume I, chapter “ML Systems”, section “The data locality invariant”.

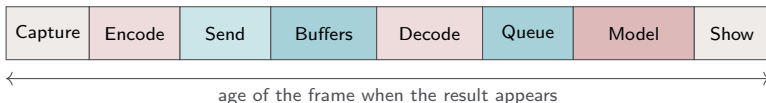
A split pipeline: filter on the camera node



- ▶ The small model runs on each frame. It must be fast and must not miss events. A false alarm costs only one transfer.
- ▶ Example: events in 5 percent of the time. The network carries 5 percent of the video: 20 times less.
- ▶ The central node runs the large model only on the frames that matter. It can serve more cameras.
- ▶ The XIAO can be such a filter: a small image model of Day 5 on the kit, and a frame to the Raspberry Pi only for an event.

Source: The share of 5 percent is an example.

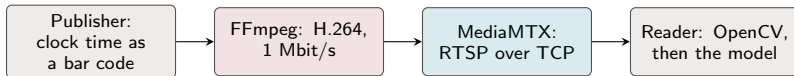
Where the age of a frame comes from



- ▶ **Capture:** the exposure and the transfer from the sensor.
- ▶ **Encode** and **decode:** B-frames and decoder threads hold frames.
- ▶ **Buffers:** the server queue, the TCP buffers, and the jitter buffer of the reader protect against changes of the network delay. Each one holds frames.
- ▶ **Queue:** frames that wait for a slow model.
- ▶ A buffer smooths the stream, and it makes each frame older. A live system keeps the buffers small.

Key point: The latency of the model is one box of eight. Measure the age of the frame at the moment of the result.

The experiment: a clock in each frame

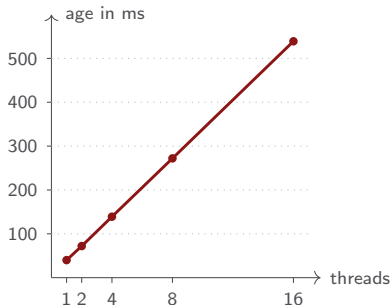


bits of the time in ms, and below them the inverted bits as a check

- ▶ 640×480 at 30 frames each second. The top rows of each frame hold the capture time in ms.
- ▶ The reader decodes the time. Age = clock time now – capture time. One computer, one clock: no clock error.
- ▶ The “model” is a pause of 0, 20, or 100 ms. Each run lasts 20 s.
- ▶ The stream does not leave the computer: the age has no network part. The lab measures with a real network.

Source: Experiment of this course: MediaMTX v1.21.1, FFmpeg 4.4, OpenCV 5.0.0, x86 work computer.

Measured: decoder threads hold frames

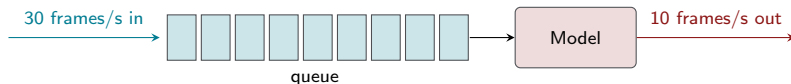


Decoder threads	Median age
1	40 ms
2	72 ms
4	139 ms
8	272 ms
16	539 ms
Default of OpenCV	538 ms

No model, reader in order.

- ▶ The H.264 decoder of FFmpeg gives each thread one frame. Each thread adds one frame time: about 33 ms.
- ▶ The default of OpenCV on 24 cores acted like 16 threads. A Raspberry Pi 5 has 4 cores: expect about 4 frames.
- ▶ For a live stream, open the stream with one decoder thread:
`cv2.CAP_PROP_N_THREADS = 1.`

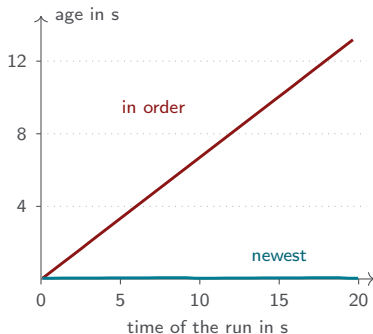
Source: Experiment of this course, x86 work computer.



- ▶ A model of 100 ms takes 10 frames each second. The stream brings 30.
- ▶ A reader that takes each frame in order adds 20 frames to its queue each second. Each frame waits longer than the frame before.
- ▶ The age grows by $1 - 10/30 = 0.67$ s each second, with no limit until a buffer is full.
- ▶ A full buffer then drops data at a place that the program does not control: a packet in the server, or a frame in the reader.

Key point: When the model is slower than the stream, the program must drop frames. Select which frames it drops.

Measured: read in order or keep the newest



Reader, model	Rate	Median	Last 2 s
In order, 20 ms	29.9	39 ms	38 ms
Newest, 20 ms	29.9	40 ms	39 ms
In order, 100 ms	9.8	6711 ms	12 734 ms
Newest, 100 ms	9.95	56 ms	42.5 ms

Rate: frames that the model took each second.
Age of the frame when the model takes it. 1 decoder thread.

- ▶ A model that is faster than the stream: the two readers are the same.
- ▶ A slow model: the reader in order shows a result for a frame of 12.7 s ago at the end. The newest reader skipped 402 of 601 frames and kept the age below 75 ms.

Source: Experiment of this course: OpenCV 5.0.0, MediaMTX v1.21.1, x86 work computer.

Method	Effect
Keep the newest frame	A thread reads all frames; the model takes the last one. The lab reader does this.
A lower frame rate at the camera	10 frames each second for a model of 100 ms: less encoder work and fewer bits
A second, smaller stream	MediaMTX can send a second stream of the same camera with its own size and rate
Skip a fixed number of frames	Simple, but the queue still grows when the model time changes

- ▶ A result for an old frame is a wrong result for a moving object. A person who walks at 1.4 m/s moves 9.4 m in 6.7 s.
- ▶ A dropped frame is a choice of the program. A full buffer drops data with no choice.
- ▶ Report two numbers: the frames each second that the model takes, and the age of those frames.

Source: MediaMTX documentation, “Raspberry Pi Cameras” (MIT). The walking speed is an example.

Place	Setting	Effect
XIAO camera driver	<code>fb_count</code> , <code>grab_mode</code>	<code>CAMERA_GRAB_WHEN_EMPTY</code> : the first <code>fb_count</code> frames can be old
H.264 encoder	B-frames	Hold later frames; the lab uses none
MediaMTX	<code>writeQueueSize</code>	Packets for a slow reader; drops when full
TCP	Socket buffers	Keep data for a reader that does not read
OpenCV reader	Decoder threads	One frame for each thread
Your program	A queue or the newest frame	Grows with a slow model

- ▶ The stream sketch of the XIAO uses one frame buffer (`fb_count = 1`) with `CAMERA_GRAB_WHEN_EMPTY`.
- ▶ In the experiment, the server dropped no packet in 20 s: TCP and the buffers kept the old frames, so the age grew. Over UDP, a full queue drops packets instead.

Source: `esp_camera.h` of the `esp32-camera` driver (esp32 core 3.3.12). MediaMTX documentation, “Decrease packet loss” (MIT).

- ▶ A camera records faces, number plates, clothes, places, and behaviour. Video of people is personal data in many laws, for example the GDPR of the European Union and the Personal Data Protection Law of Saudi Arabia.
- ▶ A stream can show more than its task needs: a person counter does not need the face of the person.
- ▶ Each copy is a risk: the stream on the network, a recording on a disk, a frame in a log, a test image on a laptop.

Questions before you install a camera:

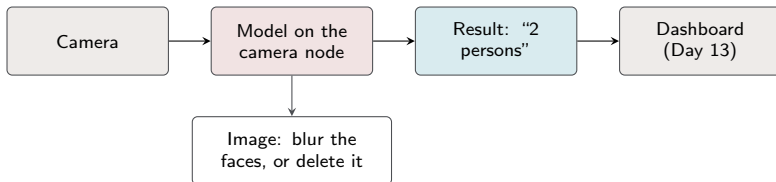
1. Which result does the task need? A count, a class, a box, or an image?
2. Where does each image go, and who can see it?
3. How long does each copy stay? Who deletes it?
4. Do the people know about the camera?

Key point: Collect the least data that the task needs. An edge device can send the result and keep the image.

Risk	The lab setup	A product
Anybody on the network watches	No password: each device on edgeai-lab can open the stream	User names and passwords in MediaMTX
Somebody reads the packets	RTSP and RTP with no encryption	RTSPS, and SRTP for the packets, with a TLS certificate
Images leave the site	A closed lab network	Results only; images on request
Old recordings	None	A fixed time, then delete

- ▶ The lab network is closed, and the cameras show the lab. This is why the lab needs no password.
- ▶ Encryption costs CPU time on the camera node. Measure it, like the encoder.
- ▶ The MJPEG stream of the XIAO has no password and no encryption. Use it only on a closed network.

Source: MediaMTX documentation, “RTSP-specific features” (section “Encryption”) and “Authentication” (MIT).



- ▶ **Send results, not images.** The detector runs where the image is made.
- ▶ **Change the image before it leaves.** Blur faces and number plates on the camera node, then stream.
- ▶ **Store events, not video.** Keep a short clip for an event, and delete it after a fixed time.
- ▶ **Limit the access.** Passwords for the stream, a separate network for the cameras, a log of each access.

Key point: The place of the model is also a privacy decision. An image that never leaves the device cannot leak from the network.

System	Place	Network	Day
XIAO: keyword or image model on the kit	Camera node	A class, later over MQTT	Days 5 and 12
Raspberry Pi: camera and detector	Camera node	Results, a stream for a person	Days 8 and 11
Raspberry Pi stream to a laptop	Central node	H.264 over RTSP	Day 11, Part C
XIAO camera to a Raspberry Pi or a laptop	Central node	MJPEG over HTTP	Day 11, Part D

- ▶ Published times: an image model on the kit needs 81 ms of pre-processing and 205 ms of inference at 96×96 pixels. A YOLO11n model at 320 pixels needs about 80 ms on the Raspberry Pi 5 with NCNN.
- ▶ The lab measures the same detector in two places: on the stream of the Raspberry Pi, and on the stream of the XIAO.

Source: *Machine Learning Systems*, kit lab “Image Classification”.
Edge AI Engineering, “Computer Vision Applications with YOLO”.

Exercise: select the inference location

For each system, select A (model on the camera node), B (model on a central node), or C (split). Give two reasons: one number and one other reason.

1. **Parking site.** 8 cameras count the free places each minute. The site has an internet link of 5 Mbit/s. No image of a car may leave the site.
2. **Doorbell.** A camera on a battery. It must show a video on the phone of the owner when a person rings. Most of the day, nothing happens.
3. **Factory line.** One camera checks each part for a defect. The belt must stop within 100 ms. A strong computer stands in the same room, with a cable to the camera.

Use the bit rates of Part 2 (2.62 Mbit/s for H.264) and the frame ages of this part.

Work with your neighbour for 8 minutes.

1. **A, or B with a node on the site.** 8 H.264 streams need $8 \times 2.62 = 21$ Mbit/s: 4 times the link. A count each minute is a few bytes. The images must stay on the site.
2. **C.** A small model or a motion test on the camera wakes the radio and the stream only for a person. The battery cannot send video all day. The video goes to the phone, with encryption.
3. **B is possible, A is safer.** A stream with 1 decoder thread had an age of 40 ms on one computer; the cable, the model, and the stop signal must fit into the other 60 ms. With 4 decoder threads (139 ms), B fails. A model on the camera node removes the video path.

Key point: Write the time budget and the bit rate first. Then select the place, and add the privacy and the power.

- ▶ TCP gives a reliable, ordered byte stream and resends lost data. UDP sends single datagrams with no resend. Video and time data often use UDP, because an old packet has no value.
- ▶ Bandwidth, latency, and jitter are three different numbers. Measure them with iperf3 and ping, and find the addresses and the open ports with ip and ss.
- ▶ An H.264 stream sends a keyframe and then only the changes, so it needs much less bandwidth than MJPEG. RTSP controls the session, RTP carries the video, and RTCP reports the quality.
- ▶ The end-to-end latency is the sum of capture, encoding, network, buffers, decoding, and inference. A reader that keeps only the newest image gives a low latency and drops old images.

Key point: Measure the age of the image that the model sees, not only the latency of the model. Select the place of the inference from the bandwidth, the latency, and the privacy of the video.

Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai), Volume I, chapter “ML Systems”, section “Edge ML: Latency and Privacy”.
- ▶ *XIAO: Big Power, Small Board*, chapter 3.4: “Implementing Wi-Fi Connection and Applications”.

Standards and papers

- ▶ RFC 3550: “RTP: A Transport Protocol for Real-Time Applications”, 2003.
- ▶ RFC 7826: “Real-Time Streaming Protocol Version 2.0”, 2016.
- ▶ “Overview of the H.264/AVC Video Coding Standard”, Wiegand, Sullivan, Bjøntegaard, and Luthra, 2003.

Tools

- ▶ MediaMTX: <https://github.com/bluenviron/mediamtx>
- ▶ FFmpeg: <https://ffmpeg.org/documentation.html>
- ▶ iperf3: <https://github.com/esnet/iperf>

This lecture adapts material from these sources:

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai. github.com/Mjrovai/EdgeML-Made-Ease-ebook and github.com/Mjrovai/EdgeML-with-Raspberry-Pi, GPL-3.0.
- ▶ *TinyML Made Easy: XIAO ESP32S3* by Marcelo Rovai. github.com/Mjrovai/XIAO-ESP32S3-Sense, Apache-2.0.
- ▶ *XIAO: Big Power, Small Board* by Lei Feng and Marcelo Rovai. github.com/Mjrovai/XIAO-Big-Power-Small-Board-ebook, GPL-3.0.

Material from a source with the licence CC BY-NC-SA 4.0 keeps that licence.